Single-Feature & Multi-Feature Regression

Gradient Descent • Numerical • MATLAB + Python (Tutorial)

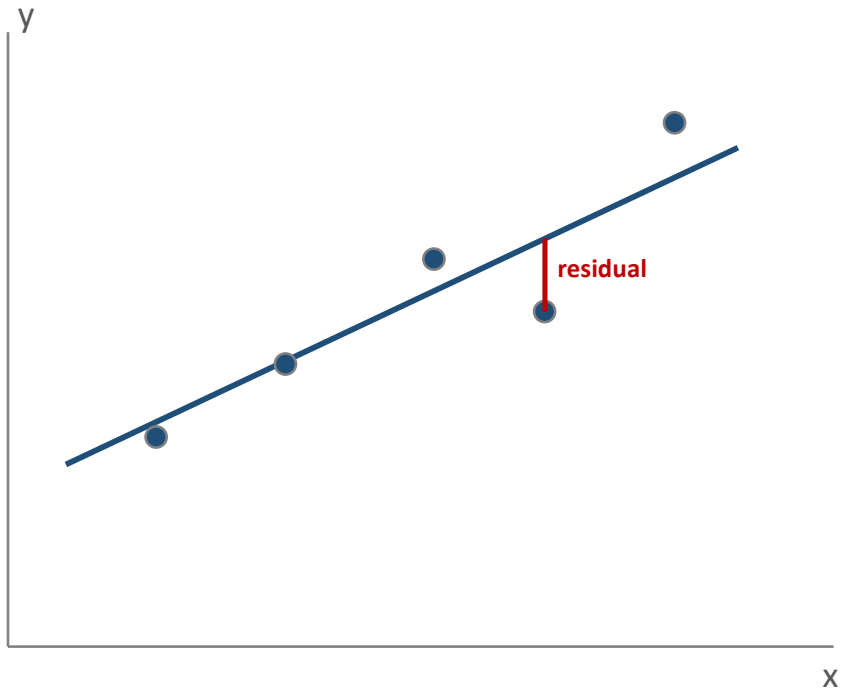
Presented by

DIBAKAR ROY

Supervisor: Dr. Avishek Adhikary

Department of Electrical Engineering, IIT Bhilai

Quick recap: what regression is actually learning



$$\hat{y} = \beta_0 + \beta_1 x$$

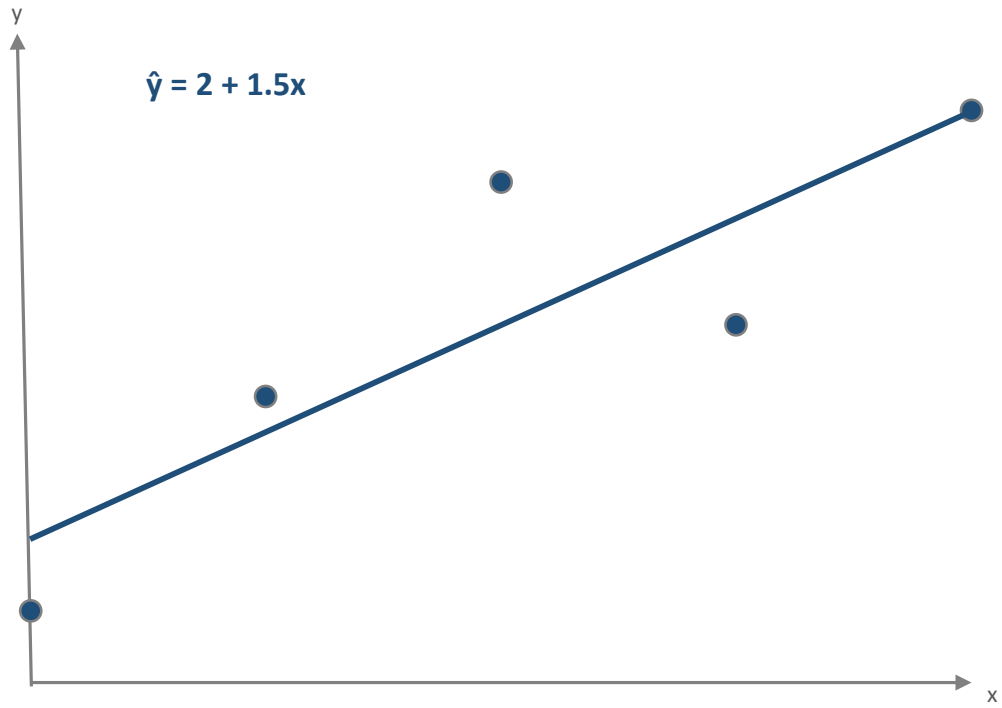
$$\text{error} = y - \hat{y}$$

β_0: intercept, starting value when x = 0
β_1: slope, change in prediction for one-unit increase in x
Residual: vertical mistake for one data point

A model earns its place only if it makes smaller mistakes than a naive guess.

Single-feature example: plant height from fertilizer dose

x dose (g)	0	1	2	3	4
y height (cm)	1	4	7	5	8



Worked out by hand

$\bar{x} = 2, \bar{y} = 5$

$\beta_1 = 1.5$

$\beta_0 = 2$

$\hat{y} = 2 + 1.5x$

$$\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Cost function: all the mistakes in one number

$$J(\beta_0, \beta_1) = \Sigma_i (y_i - \hat{y}_i)^2$$

(x)	Actual (y)	Predicted ($\hat{y}$)	Residual (y-$\hat{y}$)
0	1	2	-1
1	4	3.5	0.5
2	7	5	2
3	5	6.5	-1.5
4	8	8	0

Instead of looking at five separate residuals, we summarize them using one number: **7.5**.

SSE (sum of square error)= 7.5

SE = 0

Why squares:

We pick the line that makes the total squared residual as small as possible.

$$SST = \sum (y_i - \bar{y})^2$$ **SST = 30**

SST = 30: total variation present in the original data.

SSE = 7.5: variation/error that our regression model still fails to explain.

$R^2 = 1 - SSE/SST = 0.75$

least-squares regression

The regression model explains about 75% of the variation in y

Python, first demo: fitting the single-feature model

```python
import numpy as np

x = np.array([0, 1, 2, 3, 4], dtype=float)
y = np.array([1, 4, 7, 5, 8], dtype=float)

X = np.c_[np.ones(len(x)), x]      # intercept + feature
beta = np.linalg.lstsq(X, y, rcond=None)[0]
yhat = X @ beta
res = y - yhat
R2 = 1 - np.sum(res**2) / np.sum((y - y.mean())**2)

print('beta0, beta1 =', beta)
print('R2 =', R2)
```

Expected output

$\beta_0 = 2.0$

$\beta_1 = 1.5$

$R^2 = 0.75$

What each line is doing

- X has a column of ones
- beta stores intercept and slope
- @ means matrix multiplication
- residuals are y − prediction

MATLAB, first demo: same calculation, same answer

```
x = [0; 1; 2; 3; 4];
y = [1; 4; 7; 5; 8];

X = [ones(size(x)) x];      % intercept + feature
beta = X \ y;               % least-squares solution

yhat = X * beta;
res = y - yhat;
R2 = 1 - sum(res.^2) / sum((y - mean(y)).^2);

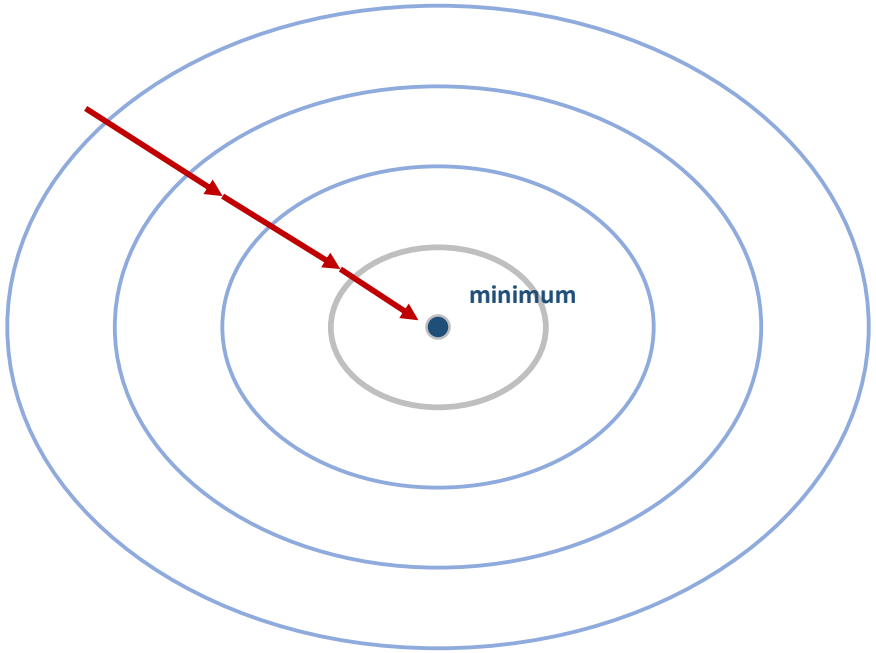
disp('beta0, beta1, R2')
disp([beta; R2])
```

Line for line against Python

Python	MATLAB
np.c_[ones, x]	[ones(size(x)) x]
X @ beta	X * beta
np.linalg.lstsq	X \ y
res**2	res.^2

Careful with X\y. It is not division. MATLAB is solving the least-squares system.

The intuition: walking downhill in a bowl



gradient = direction of steepest increase

minus gradient = downhill direction

The five steps

1. Start with β
2. Compute predictions
3. Compute gradient
4. Update β
5. Repeat until J stops decreasing

One update by hand (LMS / SGD step)

Given	value
x	1
y	2
start β_0, β_1	0, 1
α	0.1

$$\hat{y} = \beta_0 + \beta_1 x$$

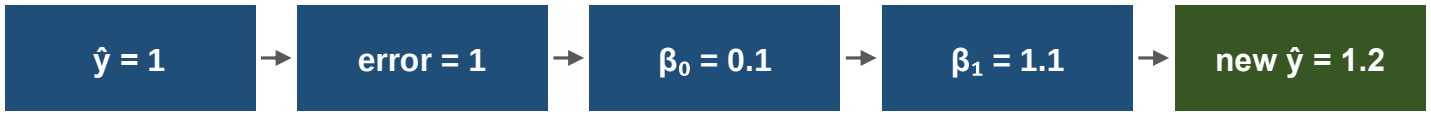
First Iteration

$$\hat{y} = 0 + (1)(1) = 1$$

$$\beta_j \leftarrow \beta_j + \alpha\,(y - \hat{y})\,x_j, \quad \text{with } x_0 = 1$$

$$\beta_0^{new} = \beta_0 + \alpha(y - \hat{y})x_0$$
$$= 0 + (0.1)(1)(1)$$
$$= \boxed{0.1}.$$

$$\beta_1^{new} = \beta_1 + \alpha(y - \hat{y})x$$
$$= 1 + (0.1)(1)(1)$$
$$= \boxed{1.1}.$$



One small update drops the error from 1.00 to 0.80.
That is all learning is here.

Learning rate α: the one setting that can wreck everything

α	What happens on our 5-point dataset
0.50	J: 86 → 9431 → 1.1 million → diverges
0.10	diverges later
0.05	converges to $\beta_0 \approx 2.20$, $\beta_1 \approx 0.60$ in lesson data
0.01	converges slowly

What to check

Plot J against iteration.

If J increases, α is too large.

Cut α by a factor of 10 and restart.

Student	J pattern	Diagnosis
A	86 → 8.2 → 5.2	roughly right
B	86 → 85.6 → 84.4	too small
C	86 → 9431 → 1.5×10^{10}	too large

There is no universal best value of α. A value like 0.1 may work very well for one dataset and fail for another, depending on feature scaling, the loss definition, and the optimization method.

Batch, Mini-batch and SGD

same update rule • different data per step

Three choices for when to update the model parameters

Batch GD

Update after



all n examples

→

Mini-batch

Update after



k examples

→

SGD

Update after



1 example

	Batch GD	Mini-batch	SGD
Path	smooth	slightly noisy	noisy zig-zag
Memory	whole dataset	one batch	one example
Practice	**rarely**	**almost always**	**sometimes**

more stable gradient ⟷ more frequent updates

Key idea Batch is easiest to explain. SGD is easiest to do by hand. Mini-batch is the practical default because hardware handles small groups efficiently.

Python, second demo: gradient descent written out

```
beta = np.zeros(2)
alpha = 0.05
n = len(y)

for k in range(200):
    yhat = X @ beta
    error = y - yhat
    grad = -(2/n) * (X.T @ error)
    beta = beta - alpha * grad

    if k % 50 == 0:
        J = np.sum(error**2)
        print(k, J, beta)

print('final beta =', beta)
```

First Iteration

$$\beta_0 = 0.5, \qquad \beta_1 = 1.3$$

Final Iteration

```
final beta = [1.99693534 1.50107503]
```

$$J(\beta_0, \beta_1) = \frac{1}{n}\sum_{i=1}^{n}(y_i - (\beta_0 + \beta_1 x_i))^2$$

Here,

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$ And $$\hat{y} = X\beta$$

$$J(\beta) = \frac{1}{n}(y - X\beta)^T(y - X\beta)$$

$$\nabla J = -\frac{2}{n}X^T(y - X\beta)$$

$$\beta \leftarrow \beta - \alpha\, grad$$

$$\text{prediction} \rightarrow \text{error} \rightarrow \text{gradient} \rightarrow \text{update}$$

MATLAB, second demo: gradient descent written out

```
beta = zeros(2,1);
alpha = 0.05;
n = length(y);

for k = 1:200
    yhat = X * beta;
    error = y - yhat;
    grad = -(2/n) * (X' * error);
    beta = beta - alpha * grad;

    if mod(k,50) == 0
        J = sum(error.^2);
        fprintf('%d  %.4f  %.4f %.4f\n', k, J, beta(1), beta(2));
    end
end

disp(beta)
```



The loop is the whole lesson. Every library hides this loop. Here you get to watch it run.

When to stop, in real code

- fixed iterations
- very small change in J
- very small gradient

More features: same idea, wider X

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

row	1	x_1	x_2
sample 1	1	−1	−1
sample 2	1	−1	+1
sample 3	1	+1	−1

Every row = one observation
Every column = one parameter
The first column of 1s learns β_0

$$\hat{y} = X\beta$$

$$X = \begin{bmatrix} 1 & x_1 & x_2 \\ 1 & x_1 & x_2 \\ 1 & x_1 & x_2 \end{bmatrix} \qquad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$$\hat{y} = X\beta = \begin{bmatrix} 1 & -1 & -1 \\ 1 & -1 & +1 \\ 1 & +1 & -1 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$$\hat{y} = \begin{bmatrix} \beta_0 - \beta_1 - \beta_2 \\ \beta_0 - \beta_1 + \beta_2 \\ \beta_0 + \beta_1 - \beta_2 \end{bmatrix}$$

The mistake everyone makes once: leaving out the column of ones.

A worked multi-feature example with coded levels

run	x_1 temp	x_2 air	y yield
1	−1	−1	14
2	−1	+1	16
3	+1	−1	16
4	+1	+1	22

$$X = \begin{bmatrix} 1 & -1 & -1 \\ 1 & -1 & +1 \\ 1 & +1 & -1 \\ 1 & +1 & +1 \end{bmatrix} \quad y = \begin{bmatrix} 14 \\ 16 \\ 16 \\ 22 \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

$$J(\beta) = (y - X\beta)^T (y - X\beta)$$

$$\nabla J = -2X^T (y - X\beta)$$

$$\beta = (X^T X)^{-1} X^T y \qquad \text{When } \nabla J = 0$$

$$X^T X = 4I \qquad (X^T X)^{-1} = \frac{1}{4} I$$

$$X^T y = \begin{bmatrix} 68 \\ 8 \\ 8 \end{bmatrix}$$

Model: $\hat{y} = 17 + 2x_1 + 2x_2$
$R^2 \approx 0.889$

Python, third demo

```
Xraw = np.array([[-1,-1], [-1,1], [1,-1], [1,1]], dtype=float)
y = np.array([14, 16, 16, 22], dtype=float)
X = np.c_[np.ones(len(y)), Xraw]

# least-squares solution
beta = np.linalg.lstsq(X, y, rcond=None)[0]
yhat = X @ beta
error = y - yhat

SSE = np.sum(error**2)
SST = np.sum((y - y.mean())**2)
R2 = 1 - SSE/SST

print('beta0, beta1, beta2 =', beta)
print('R2 =', R2)
```

Batch Gradient Descent

Run	x_1	x_2	y
1	-1	-1	14
2	-1	$+1$	16
3	$+1$	-1	16
4	$+1$	$+1$	22

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$J = \frac{1}{n}\sum_{i=1}^{n}\left[y_i - \beta_0 - \beta_1 x_{1i} - \beta_2 x_{2i}\right]^2$$

$$\beta_j \leftarrow \beta_j - \alpha\frac{\partial J}{\partial \beta_j}$$

Let, $e_i = y_i - \hat{y}_i$

$$J(\beta) = \frac{1}{n}\sum_{i=1}^{n} e_i^2$$

Here, $n = 4$

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{n}\sum 2e_i\frac{\partial e_i}{\partial \beta_0}$$

Where, $\frac{\partial e_i}{\partial \beta_0} = -1$

$$\beta^{new} = \beta^{old} - \alpha\nabla J.$$

$$\nabla J = \begin{bmatrix} -\frac{2}{n}\sum e_i \\ -\frac{2}{n}\sum e_i x_{1i} \\ -\frac{2}{n}\sum e_i x_{2i} \end{bmatrix}$$

$$\frac{\partial J}{\partial \beta_0} = -\frac{2}{n}\sum e_i \qquad \frac{\partial J}{\partial \beta_1} = -\frac{2}{n}\sum e_i x_{1i}$$

$$\frac{\partial J}{\partial \beta_2} = -\frac{2}{n}\sum e_i x_{2i}$$

$$\begin{bmatrix}\beta_0 \\ \beta_1 \\ \beta_2\end{bmatrix}_{new} = \begin{bmatrix}0 \\ 0 \\ 0\end{bmatrix} - 0.25\begin{bmatrix}-34 \\ -4 \\ -4\end{bmatrix}$$

$$\nabla J = -\frac{2}{4}\begin{bmatrix}68 \\ 8 \\ 8\end{bmatrix}$$

$$\beta = \begin{bmatrix}8.5 \\ 1 \\ 1\end{bmatrix}$$

Multi-feature GD: component-wise Python code

```
import numpy as np

# data
Xraw = np.array([[-1,-1],
                 [-1, 1],
                 [ 1,-1],
                 [ 1, 1]], dtype=float)

y = np.array([14, 16, 16, 22], dtype=float)

# add column of 1s for beta0
X = np.c_[np.ones(len(y)), Xraw]

# initial values
beta = np.zeros(3)
alpha = 0.25
n = len(y)

# gradient descent
for k in range(20):
    yhat = X @ beta
    error = y - yhat

    grad0 = -(2/n) * np.sum(error)
    grad1 = -(2/n) * np.sum(error * Xraw[:,0])
    grad2 = -(2/n) * np.sum(error * Xraw[:,1])
    grad = np.array([grad0, grad1, grad2])

    beta = beta - alpha * grad
    J = np.mean(error**2)
    print(k+1, J, beta)

print('final beta =', beta)
```

MATLAB, third demo: closed form and one pass of SGD

```
Xraw = [-1 -1; -1 1; 1 -1; 1 1];
y = [14; 16; 16; 22];
X = [ones(size(y)) Xraw];

% check closed-form answer
beta_closed = X \ y;
disp(beta_closed)

% one pass of SGD
beta = zeros(3,1);
alpha = 0.01;
for i = 1:length(y)
    yhat = X(i,:) * beta;
    error = y(i) - yhat;
    beta = beta + alpha * error * X(i,:)';
    disp(beta')
end

disp('after one pass')
disp(beta)
```

Notation worth pausing on

symbol	meaning
X(i,:)	one row
X(i,:)'	column vector
*	matrix product
.*	elementwise

The error you will hit: dimension mismatch.
Check row against column before you run it.